SIMATS ENGINEERING

CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1709

Register Number	X. Joshua
Name	192312188

COURSE OUTCOME COVERED

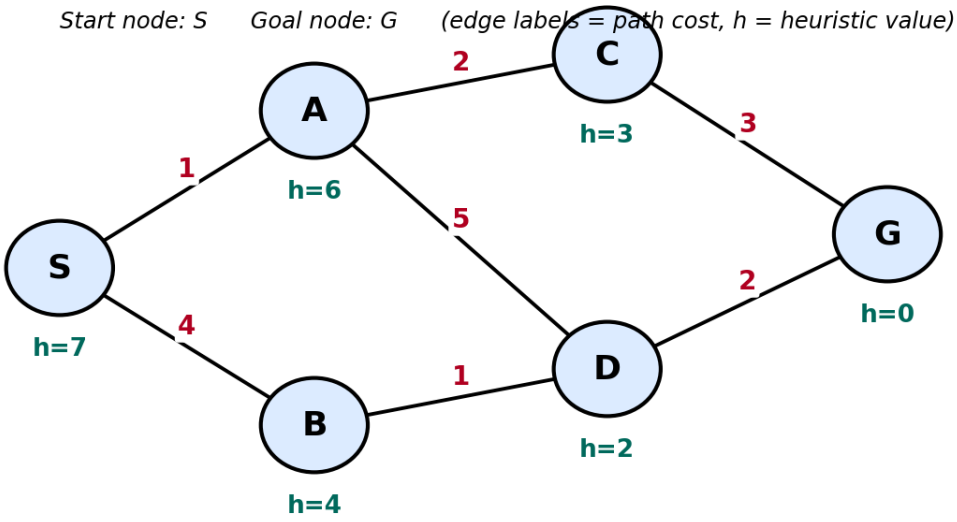
CO2: Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks: 50**

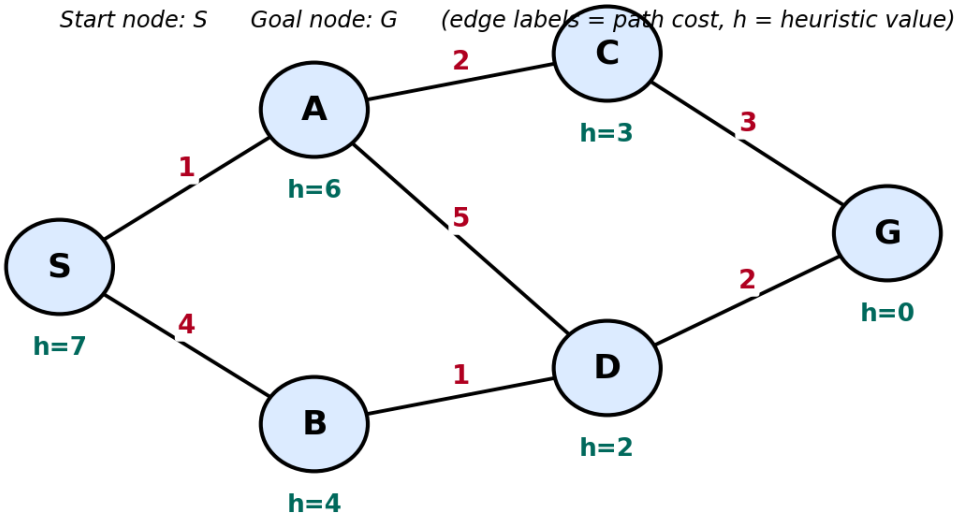
Q1.

Consider a suitable state-space graph (with heuristic values h(n) assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.



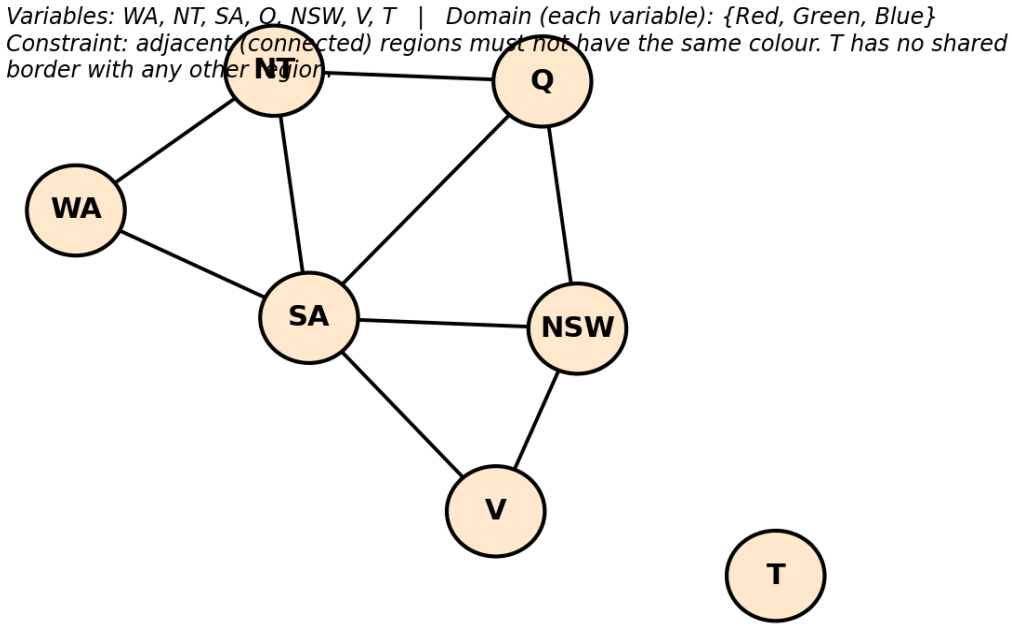
Q2.

For a suitable weighted state-space graph (with edge costs and heuristic values h(n)) of your choice, perform a dry run of A* Search from the start node to the goal node. Show the g(n), h(n) and f(n) values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.



Q3.

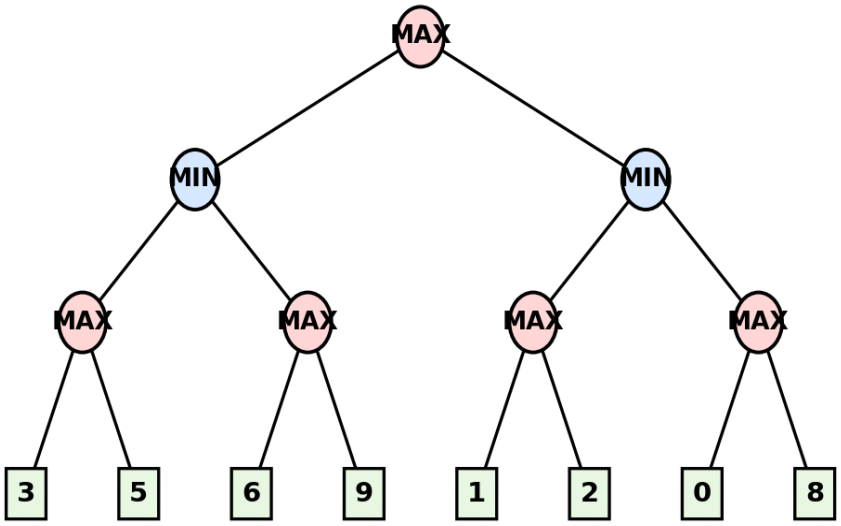
Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.



Q4.

For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

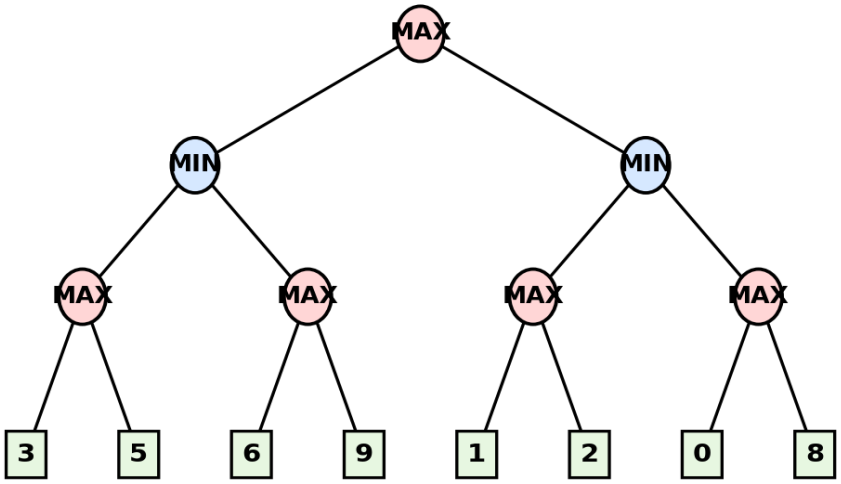
Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Q5.

Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Code of Conduct:

I X.Joshua-(192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	**10**
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	**10**
Accuracy of Computations (g(n), h(n), f(n) values, minimax backed-up values, or α–β updates)	All computed values are accurate and consistent throughout the dry run.	Minor calculation errors are present but do not significantly affect the final outcome.	Multiple calculation errors are present that affect the correctness of intermediate steps.	Calculations are mostly incorrect or missing.	**10**
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	**10**
Clarity of Presentation & Justification (trace tables/trees/diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation.	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	**10**
				Total	50

Answer 1: Greedy Best-First Search — Dry Run

Problem Formulation & Setup

State-space graph: Start node S, goal node G, and intermediate nodes A, B, C, D. The graph is undirected, and each node carries a heuristic value h(n) representing the estimated straight-line distance to the goal G. Greedy Best-First Search uses only h(n) — the actual path cost g(n) is never consulted — so only connectivity and h(n) need to be defined here.

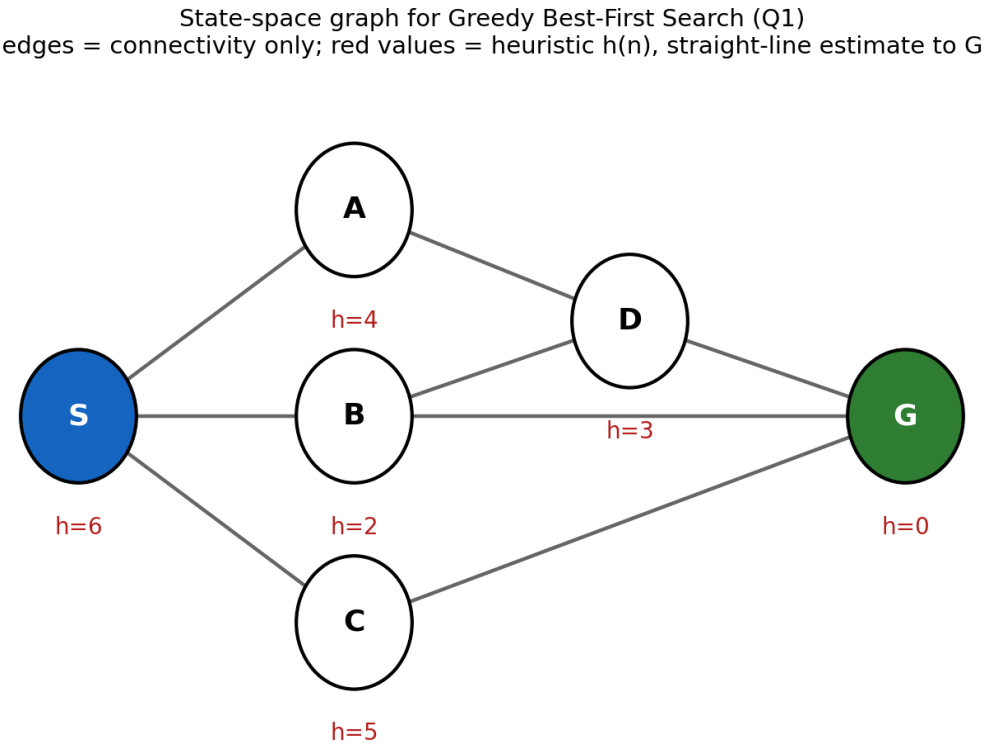


Fig. 1 — State-space graph with heuristic values h(n) used for the GBFS dry run.

Node	S	A	B	C	D	G
h(n)	6	4	2	5	3	0

Edges (connectivity): S–A, S–B, S–C, A–D, B–D, B–G, C–G, D–G.

Algorithm Procedure & Dry Run Trace

Greedy Best-First Search maintains a frontier (open list) ordered purely by ascending h(n). At each step, the node with the lowest h(n) is popped and expanded; its unvisited neighbours are pushed onto the frontier. The process stops the moment the goal node is popped for expansion.

Step	**Node Expanded**	**h(n)**	**Successors Generated (h)**	**Frontier after this step (sorted by h)**
1	— (initial)	—	—	[S(6)]
2	S	6	A(4), B(2), C(5)	[B(2), A(4), C(5)]
3	B	2	D(3), G(0)	[G(0), D(3), A(4), C(5)]
4	G	0	GOAL — stop expansion	search terminates

Final Outcome

Path obtained: S → B → G (G is popped as soon as it enters the frontier, because it has the lowest possible heuristic value, h = 0).

Strengths and Weaknesses under Uncertainty

- Strength — Speed: GBFS reached the goal after expanding just two nodes (S and B), because it always chases the neighbour that 'looks' closest, which is ideal when a fast decision is needed.
- Strength — Low memory: only h-ordering is tracked, no cumulative cost bookkeeping is required.
- Weakness — Not optimal: GBFS never checks whether the path S–B–G is actually the cheapest one; if edge S–B or B–G had a very high real-world cost (e.g., a flooded/blocked road in a delivery context), GBFS would still take it purely because h(B) and h(G) look good.
- Weakness — Not complete in general graphs: without cycle/visited-node checking, GBFS can loop indefinitely if the heuristic misleads it back toward already-visited regions.
- Weakness — Sensitive to heuristic quality: an inaccurate h(n) directly produces a poor route, since cost is never used as a correction.

Answer 2: A* Search — Dry Run

Problem Formulation & Setup

Same node set (S, A, B, C, D, G), now with the graph weighted by real edge costs, and heuristic h(n) chosen to be admissible and consistent (h(n) never overestimates the true remaining cost to G).

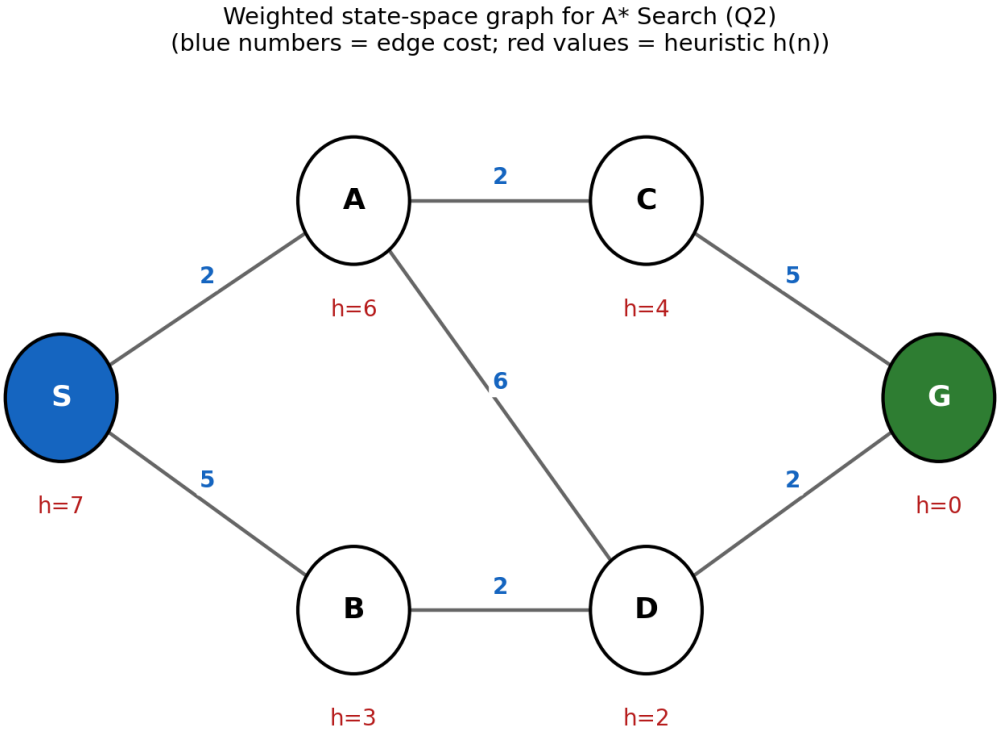


Fig. 2 — Weighted state-space graph with edge costs (blue) and heuristic values h(n) (red) used for the A dry run.*

Node	S	A	B	C	D	G
h(n)	7	6	3	4	2	0

Edges with cost: S–A (2), S–B (5), A–C (2), A–D (6), B–D (2), C–G (5), D–G (2).

Algorithm Procedure & Dry Run Trace (f(n) = g(n) + h(n))

A* keeps a priority frontier ordered by ascending f(n). At each step the node with lowest f is expanded; if a shorter g-path to an already-generated node is found, its g/f values are updated (this happens for node D below). The search halts once the goal is actually popped for expansion (not merely generated), guaranteeing optimality for an admissible, consistent heuristic.

Step	Node Expanded	g / h / f	Successors generated / updated (g, h, f)	Frontier after step (by f)
1	S	0 / 7 / 7	A(2,6,8), B(5,3,8)	A(f8), B(f8)
2	A	2 / 6 / 8	C(4,4,8), D(8,2,10)	B(f8), C(f8), D(f10)
3	B	5 / 3 / 8	D updated: 7,2,9 (better than 8,2,10 via A)	C(f8), D(f9)
4	C	4 / 4 / 8	G(9,0,9) [via C]	D(f9), G(f9)
5	D	7 / 2 / 9	G via D = (9,0,9) — same g as existing G, no improvement	G(f9)
6	G	9 / 0 / 9	GOAL popped for expansion — stop	search terminates

Final Outcome

Optimal path: S → A → C → G, with total cost = 2 + 2 + 5 = 9.

Note: An equal-cost alternative path S → B → D → G (cost = 5 + 2 + 2 = 9) also exists — this tie is expected and correct behaviour, since both paths genuinely have the same minimum cost; A* is guaranteed only to find an optimal path, not necessarily a unique one.

Answer 3: Constraint Satisfaction Problem — Map Colouring (Backtracking Search)

CSP Formulation

Problem: Colour the map of Australia so that no two neighbouring regions share the same colour (classic Map-Colouring CSP).

- Variables: WA, NT, SA, Q, NSW, V, T (the seven regions).
- Domain (each variable): {Red, Green, Blue}.
- Constraints (binary, 'not-equal'): WA≠NT, WA≠SA, NT≠SA, NT≠Q, SA≠Q, SA≠NSW, SA≠V, Q≠NSW, NSW≠V. (T shares a border with no other region, so it has no constraints — it is a free variable.)

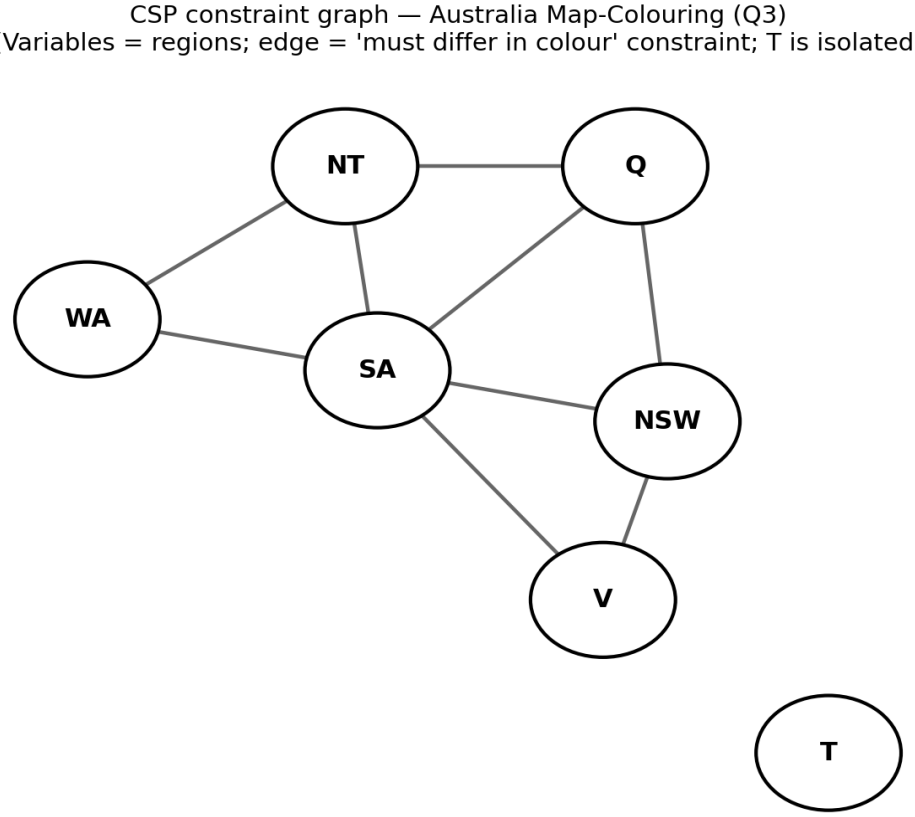


Fig. 3 — CSP constraint graph: an edge between two regions means their colours must differ.

Backtracking Search — Dry Run

Variable order used: WA, NT, SA, Q, NSW, V, T. Value order tried at each variable: Red, then Green, then Blue.

Step	Variable	Value tried	Constraint check	Result
1	WA	Red	No prior assignments to conflict with	Assigned WA=Red
2	NT	Red	WA–NT constraint: Red = Red → violated	Rejected, try next value
3	NT	Green	WA–NT: Red ≠ Green → OK	Assigned NT=Green
4	SA	Red	WA–SA: Red = Red → violated	Rejected, try next value
5	SA	Green	NT–SA: Green = Green → violated	Rejected (backtrack within domain), try next value
6	SA	Blue	WA–SA: Red≠Blue OK; NT–SA: Green≠Blue OK	Assigned SA=Blue
7	Q	Red	NT–Q: Green≠Red OK; SA–Q: Blue≠Red OK	Assigned Q=Red
8	NSW	Red	Q–NSW: Red = Red → violated	Rejected, try next value
9	NSW	Green	SA–NSW: Blue≠Green OK; Q–NSW: Red≠Green OK	Assigned NSW=Green
10	V	Red	SA–V: Blue≠Red OK; NSW–V: Green≠Red OK	Assigned V=Red
11	T	Red	No constraints on T	Assigned T=Red

Backtracking steps observed: at SA (step 4–6) two values (Red, then Green) were tried and rejected by the constraint check before Blue succeeded — this local retry-on-failure, without needing to revisit an earlier variable, is exactly the recovery mechanism backtracking search uses whenever a value violates a constraint. The same rejection-then-retry pattern appears again at NSW (step 8–9).

Final Outcome

Complete, consistent solution found: WA=Red, NT=Green, SA=Blue, Q=Red, NSW=Green, V=Red, T=Red — every adjacent pair of regions has a different colour, and no full backtrack to an earlier variable was required because 3 colours are sufficient to properly colour this constraint graph (the WA–NT–SA triangle alone forces a minimum of 3 colours).

Note: why the domain size matters

If only 2 colours were allowed instead of 3, the WA–NT–SA triangle (WA–NT, NT–SA, WA–SA are all constrained) could never be 2-coloured, since three mutually-constrained variables always need at least 3 distinct values. In that case, backtracking search would repeatedly exhaust SA's domain, backtrack to reassign NT, exhaust NT's domain, backtrack to WA, and ultimately report failure — a genuine multi-level backtrack, illustrating why an adequately sized domain is part of correct problem formulation.

Answer 4: Minimax Algorithm — Dry Run

Game Tree Setup

A two-player, zero-sum game tree, 3 plies deep, branching factor 2 throughout: the root is a MAX node with two children (B, C) which are MIN nodes; each of B and C has two MAX children (D, E and F, G respectively); each of D, E, F, G has two terminal leaves with given utility values.

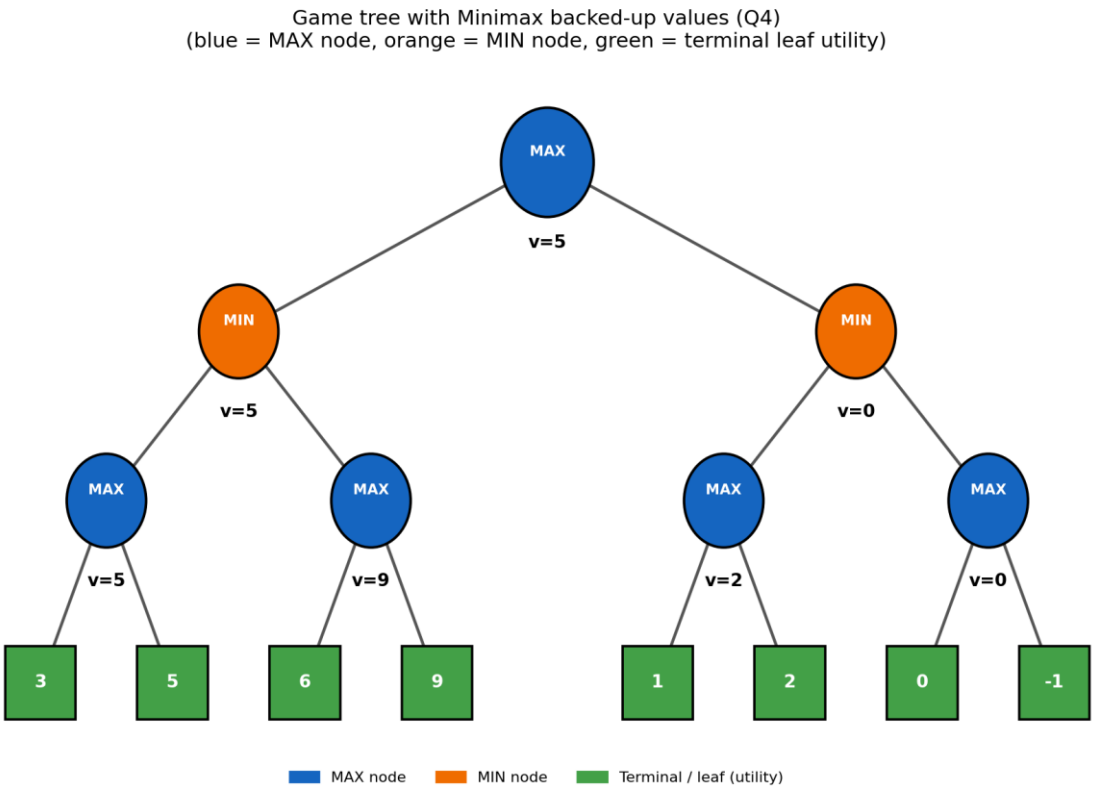


Fig. 4 — Game tree showing terminal utility values (green leaves) and the Minimax backed-up value at every internal node.

Backed-up Value Computation (bottom-up)

Node	Type	Children values	Computation	Backed-up value
D	MAX	leaves: 3, 5	max(3,5)	5
E	MAX	leaves: 6, 9	max(6,9)	9
B	MIN	D=5, E=9	min(5,9)	5
F	MAX	leaves: 1, 2	max(1,2)	2
G	MAX	leaves: 0, -1	max(0,-1)	0
C	MIN	F=2, G=0	min(2,0)	0
ROOT	MAX	B=5, C=0	max(5,0)	5

Best Move for MAX (Root) — with Justification

The root's backed-up value is 5, achieved through child B (value 5), versus child C (value 0). MAX should therefore play the move leading to B, since 5 > 0. Intuitively: if MAX moves toward C, a rational MIN opponent can always force the outcome down to 0 (by choosing G); but if MAX moves toward B, the worst the opponent can force is 5 (choosing D over E, since D=5 < E=9). Minimax always selects the move that maximizes the guaranteed (worst-case) outcome, so B is the correct, justified choice.

Answer 5: Minimax with Alpha-Beta Pruning — Dry Run

Using the same game tree as Q4, traversed depth-first, left to right, maintaining α (best value MAX can guarantee so far on the path) and β (best value MIN can guarantee so far on the path). A branch is pruned whenever $\alpha \geq \beta$ at a node, because the remaining siblings cannot change the outcome that a rational opponent would already avoid.

Step-by-step α–β Trace

Node visited	α (in)	β (in)	Action	Returns
ROOT (MAX)	-∞	+∞	Calls child B	—
B (MIN)	-∞	+∞	Calls child D	—

D (MAX)	-∞	+∞	Leaf 3 → best=3; leaf 5 → best=5. No prune (last child).	5
B (MIN) cont.	-∞	+∞	β = min(+∞,5) = 5. β>α, continue to child E with (α=-∞, β=5).	—
E (MAX)	-∞	5	Leaf 6 → best=6, α=max(-∞,6)=6. Check α≥β: 6≥5 → PRUNE remaining leaf (9) — never evaluated.	6
B (MIN) cont.	-∞	+∞	β = min(5,6) = 5. No more children.	5
ROOT cont.	-∞	+∞	α = max(-∞,5) = 5. Calls child C with (α=5, β=+∞).	—
C (MIN)	5	+∞	Calls child F with (α=5, β=+∞).	—
F (MAX)	5	+∞	Leaf 1 → best=1; leaf 2 → best=2. α stays 5 (max(5,2)=5). No prune (last child).	2
C (MIN) cont.	5	+∞	β = min(+∞,2) = 2. Check β≤α: 2≤5 → PRUNE remaining child G entirely — its leaves (0, -1) never evaluated.	2
ROOT cont.	5	+∞	α = max(5,2) = 5 (unchanged). No more children.	5

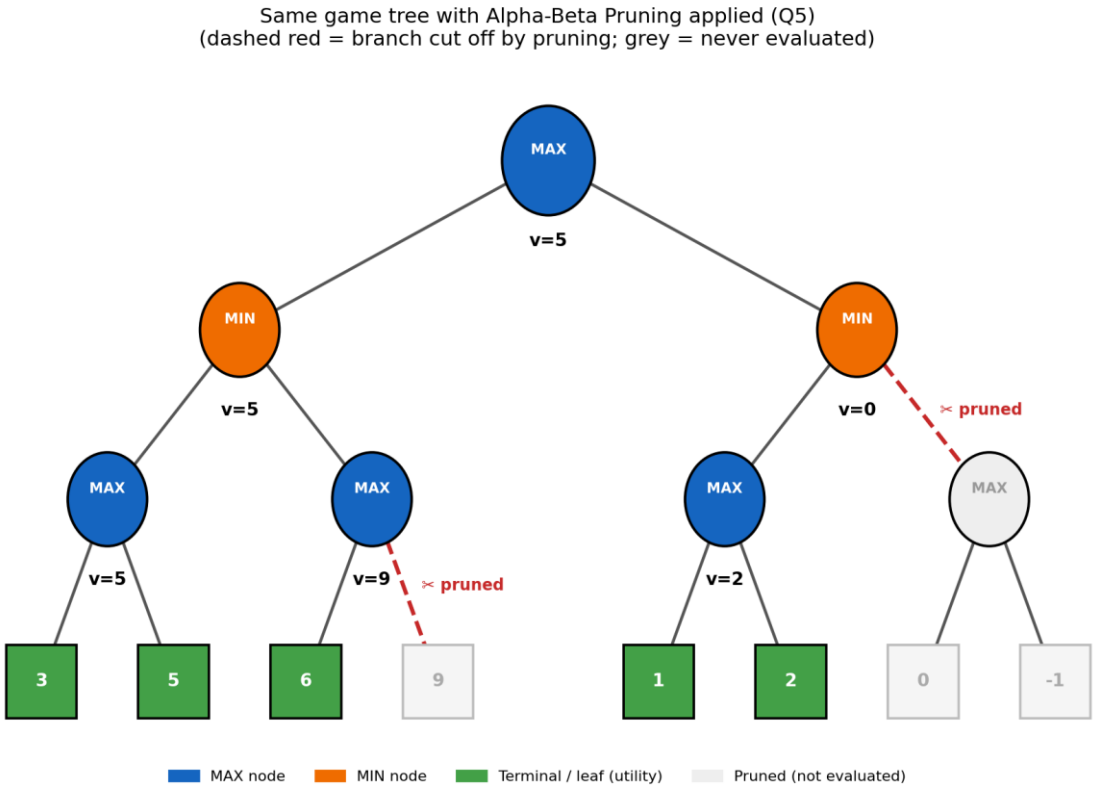


Fig. 5 — Same tree with Alpha-Beta Pruning: the dashed red branches (E's second leaf, and the entire subtree under G) are cut off and never evaluated.

Branches Pruned and Why

- At node E (MAX, child of B): after seeing leaf value 6, α becomes 6, which is ≥ the inherited β = 5 from B. This means MIN (at B) already has a better alternative (D = 5) than anything E could offer once E reaches 6 or higher, so the remaining leaf under E (value 9) is pruned — it cannot change B's decision.
- At node C (MIN): after exploring child F and obtaining 2, β becomes 2, which is ≤ the inherited α = 5 from the root. This means MAX (at the root) already has a better alternative (B = 5) than anything C could offer once C drops to 2 or lower, so the entire remaining child G (and both its leaves, 0 and -1) is pruned.

Final Best Move

The root value obtained with Alpha-Beta Pruning is 5 — identical to the value found by full Minimax in Q4 — confirming that pruning never changes the final decision, only the amount of the tree that must be searched. The best move for MAX is therefore, again, to move toward B. In this small example 3 leaf evaluations (E's second leaf, and G's two leaves) were skipped out of 8 total, illustrating how Alpha-Beta pruning reduces computation while guaranteeing the same optimal result — which is essential for a real-time game AI operating under strict time limits.